GUÍA MAESTRA DE R

Diccionario de funciones para evaluaciones

Catálogo rápido y estructurado · versión mejorada

CONTENIDO

0. CONVENCIONES DE ESTA GUÍA
1. EXPLORACIÓN Y CARGA DE DATOS
2. TIPOS DE VARIABLES Y CONVERSIÓN
3. DATOS FALTANTES (NA)
4. LIMPIEZA DE TEXTOS
5. FECHAS Y HORAS (library(lubridate))
6. MANIPULACIÓN CON DPLYR (library(dplyr))
7. TRABAJO CON DOS BASES (DFF y DFR)
8. ESTADÍSTICA DESCRIPTIVA
9. VALORES ATÍPICOS (OUTLIERS)
10. PROBABILIDADES EMPÍRICAS Y OPERADORES LÓGICOS
11. VARIABLES ALEATORIAS CONTINUAS (FUNCIÓN DE DENSIDAD)
12. MODELOS DE PROBABILIDAD (DISTRIBUCIONES)
13. GRÁFICOS (MUESTRA vs MODELO TEÓRICO)
14. CONCEPTOS CLAVE: LIMPIA vs PROLIJA
15. EJERCICIOS RESUELTOS

0. CONVENCIONES DE ESTA GUÍA

Todas las funciones de esta guía están escritas para usarse directamente sobre la base de datos con nombres fijos:

- **Una sola base:** siempre se llama `DF`.
- **Dos bases:** se llaman `DFF` y `DFR` (por ejemplo, F = primera/inicial y R = segunda/recolección; usa la que corresponda al enunciado).
- Una columna se escribe `DF$Variable` (ej. `DF$Peso`). Dentro de funciones de dplyr se escribe solo el nombre (`Peso`).
- **Regla de oro:** TODO resultado numérico se entrega con `round(..., 2)` (2 decimales). Ej. `round(mean(DF$Peso, na.rm = TRUE), 2)`.
- **Excepción:** si la respuesta solo puede ser ENTERA (conteos: filas, países, medallas) se responde EXACTA, sin decimales.
- Las filas sombreadas en amarillo y marcadas con ★ **NUEVO** son funciones añadidas a partir de los ejercicios resueltos.

Plantilla de inicio (cópiala al principio de todo script):

```
# Librerías (instalar una sola vez:
install.packages(c("readr","dplyr","tidyr","stringr","lubridate")))
library(readr)      # read_csv
library(dplyr)      # filter, mutate, select, group_by, summarise...
library(tidyr)      # separate, pivot_longer, drop_na
library(stringr)    # str_detect, str_extract, str_to_lower
library(lubridate)  # ymd_hms, hour, with_tz

# UNA base
DF <- read_csv("archivo.csv")

# DOS bases
DFF <- read_csv("archivo1.csv")
DFR <- read_csv("archivo2.csv")
```

1. EXPLORACIÓN Y CARGA DE DATOS

Función / Comando	¿Qué hace? (uso exacto)	Ejemplo con DF
`read_csv("archivo.csv")`	Carga un CSV como tibble (requiere `library(readr)`). Detecta fechas automáticamente como POSIXct.	`DF <- read_csv("Cuy.csv")`
`read.csv("archivo.csv")` ★ NUEVO	Versión base de R (sin librerías). NO convierte fechas: quedan como texto. Usar solo si el examen no pide readr.	`DF <- read.csv("Cuy.csv")`
`head(DF) / tail(DF)` ★ NUEVO	Muestra las primeras / últimas 6 filas.	`head(DF, 10)`
`str(DF)`	Estructura interna: nombre de cada variable, su tipo (chr, num, int, logi, POSIXct) y primeros valores.	`str(DF)`
`glimpse(DF)` ★ NUEVO	Igual que `str` pero más ordenado (dplyr).	`glimpse(DF)`
`summary(DF)` ★ NUEVO	Resumen de cada columna: mín., cuartiles, media, máx. y cantidad de NA.	`summary(DF)` `summary(DF$Peso)`
`names(DF)` ★ NUEVO	Lista los nombres de las columnas.	`names(DF)`
`nrow(DF)`	Número de filas (observaciones).	`nrow(DF)  # 8755`
`ncol(DF)`	Número de columnas (variables).	`ncol(DF)  # 13`
`dim(DF)`	Dimensiones: [filas, columnas].	`dim(DF)  # 8755 13`

Función / Comando	¿Qué hace? (uso exacto)	Ejemplo con DF
`sapply(DF, class)` ★ NUEVO	Devuelve el tipo de CADA columna. Útil para clasificar variables.	`sapply(DF, class)`
`sum(sapply(DF, is.numeric))`	Cuenta columnas que R guarda como número. ¡OJO! Un código numérico (ej. `Equipo` = 1,2,3) cuenta como número para R pero es categórico. Corrige a mano.	`sum(sapply(DF, is.numeric))` `# 6 en Cuy, pero Equipo es categ.`
`unique(DF$Var)` ★ NUEVO	Muestra los valores distintos de una variable.	`unique(DF$Raza)`
`n_distinct(DF$Var)` ★ NUEVO	Cuenta cuántos valores distintos tiene una variable (dplyr). Equivale a `length(unique(x))`.	`n_distinct(DF$Tiempo)  # 8574`
`length(unique(DF$Var))` ★ NUEVO	Cuántos valores distintos hay (ej. cuántos PAÍSES participan). Forma simple sin dplyr.	`length(unique(DF$País))  # 16`
`sum(DF$Var)` ★ NUEVO	Suma total de una columna numérica (ej. total de medallas). Con NA: na.rm = TRUE.	`sum(DF$Primeros)  # 27`
`colSums(DF[, c(...)])` ★ NUEVO	Suma varias columnas a la vez.	`colSums(DF[, c("Primeros", "Segundos", "Terceros")])`
`sapply(DF, n_distinct)` ★ NUEVO	Cantidad de valores distintos por columna. Ayuda a decidir si una variable es categórica (pocos) o continua (muchísimos).	`sapply(DF, n_distinct)`

2. TIPOS DE VARIABLES Y CONVERSIÓN

Clasificación conceptual (lo que se pregunta en las evaluaciones):

Tipo	Cómo reconocerla	Ejemplo (base Cuy)
Categórica nominal	Categorías SIN orden. Incluye textos, códigos numéricos (IDs, equipos) y lógicas TRUE/FALSE.	Raza, Sexo, Color, Reporte, Equipo, Error
Categórica ordinal	Categorías CON orden natural.	Talla (flaco < sano < gordo < obeso)
Fecha / fecha-hora	Clase Date o POSIXct.	Fecha
Numérica continua	Puede tomar cualquier valor en un intervalo; casi todos sus valores son distintos.	Tiempo
Numérica discreta	Valores contables o REDONDEADOS al registrar (enteros, 2 decimales). Una continua por naturaleza queda discretizada al medirse.	Peso, Largo, Cintura, Edad

Función / Comando	¿Qué hace? (uso exacto)	Ejemplo con DF
`factor(DF$Var)`	Convierte en categórica nominal. Evita que R promedie IDs, años o códigos.	`DF$Equipo <- factor(DF$Equipo)`
`factor(x, levels=..., ordered=TRUE)` ★ NUEVO	Convierte en categórica ORDINAL indicando el orden de las categorías.	`DF$Talla <- factor(DF$Talla, levels = c("flaco","sano","gordo","obeso"), ordered = TRUE)`
`levels(DF$Var)` ★ NUEVO	Muestra las categorías de un factor (y su orden).	`levels(DF$Talla)`

Función / Comando	¿Qué hace? (uso exacto)	Ejemplo con DF
as.numeric(DF$Var)	Fuerza a numérico. Si hay texto, lo vuelve NA (revisa antes).	`DF$Peso <- as.numeric(DF$Peso)`
as.integer(DF$Var) ★ NUEVO	Fuerza a entero (útil para Edad, Largo).	`DF$Edad <- as.integer(DF$Edad)`
as.character(DF$Var) ★ NUEVO	Fuerza a texto.	`as.character(DF$Equipo)`
as.logical(DF$Var) ★ NUEVO	Convierte "TRUE"/"FALSE" (texto) en lógico.	`DF$Error <- as.logical(DF$Error)`
class(DF$Var) ★ NUEVO	Tipo de UNA variable.	`class(DF$Fecha)  # POSIXct`

3. DATOS FALTANTES (NA)

Función / Comando	¿Qué hace? (uso exacto)	Ejemplo con DF
is.na(DF$Var)	TRUE en las celdas vacías (NA).	`sum(is.na(DF$Peso))  # 1340`
colSums(is.na(DF))	Cantidad de NA por columna.	`colSums(is.na(DF))`
sum(complete.cases(DF))	Filas 100% completas (muestra efectiva).	`sum(complete.cases(DF))  # 4007`
sum(!complete.cases(DF)) ★ NUEVO	Observaciones INCOMPLETAS: filas con al menos un NA. El ! invierte TRUE/FALSE.	`sum(!complete.cases(DF))  # 4748`
mean(! complete.cases(DF))*100 ★ NUEVO	Porcentaje de filas incompletas.	`round(mean(!complete.cases(DF))*100, 2)` `# 54.23 %`
na.omit(DF)	¡PELIGRO! Borra TODA la fila si tiene un solo NA. Solo si el examen lo pide.	`DF_sinNA <- na.omit(DF)`
drop_na(DF, Var) ★ NUEVO	Borra filas con NA solo en las columnas indicadas (tidyr). Más seguro que na.omit.	`DF %>% drop_na(Peso)`
na.rm = TRUE	¡SEGURO! Argumento dentro de mean, sd, median, sum, quantile... para ignorar NA sin borrar filas.	`round(mean(DF$Peso, na.rm = TRUE), 2)`

4. LIMPIEZA DE TEXTOS

Función / Comando	¿Qué hace? (uso exacto)	Ejemplo con DF
gsub("buscar", "nuevo", x)	Reemplaza TODAS las coincidencias de un texto. Sirve para quitar letras de variables numéricas.	`DF$Peso <- as.numeric(` `  gsub(" g", "", DF$Peso))`
sub("buscar", "nuevo", x) ★ NUEVO	Igual que gsub pero solo la PRIMERA coincidencia.	`sub(".*Status: ", "", DF$Reporte)`
tolower(x) / toupper(x) ★ NUEVO	Pasa a minúsculas / mayúsculas (unifica "Macho", "MACHO").	`DF$Sexo <- tolower(DF$Sexo)`
trimws(x) ★ NUEVO	Quita espacios al inicio y al final.	`DF$Raza <- trimws(DF$Raza)`
str_detect(x, "patrón") ★ NUEVO	TRUE si el texto contiene el patrón (stringr).	`sum(str_detect(DF$Reporte, "ERR"))` `# 68`
str_extract(x, "regex") ★ NUEVO	Extrae la parte del texto que coincide con el patrón.	`str_extract(DF$Reporte,` `  "(?<=Status: )\\w+")  # OK / ERR`

Función / Comando	¿Qué hace? (uso exacto)	Ejemplo con DF
separate(DF, col, into, sep) ★ NUEVO	Divide una columna en varias (tidyr). Arregla variables NO prolijas como Reporte.	`DF %>% separate(Reporte, into = c("SerNo","FechaRep","Status"), sep = " - ")`
%in% ★ NUEVO	¿El valor pertenece a una lista? Útil para validar contra la libreta de códigos.	`sum(!DF$Raza %in% c("abisinio", "andino","peruano"))`

5. FECHAS Y HORAS (library(lubridate))

Función / Comando	¿Qué hace? (uso exacto)	Ejemplo con DF
ymd_hms(x) ★ NUEVO	Convierte texto "AAAA-MM-DD HH:MM:SS" en fecha-hora POSIXct.	`DF$Fecha <- ymd_hms(DF$Fecha)`
as.POSIXct(x, tz="UTC") ★ NUEVO	Conversión a fecha-hora en R base.	`as.POSIXct(DF$Fecha, tz = "UTC")`
as.Date(x) ★ NUEVO	Se queda solo con la fecha (sin hora).	`as.Date(DF$Fecha)`
year(), month(), day(), hour(), minute() ★ NUEVO	Extrae una parte de la fecha.	`table(hour(DF$Fecha))`
with_tz(x, "America/Lima") ★ NUEVO	Muestra la misma hora en otra zona horaria (UTC → Lima = -5h).	`with_tz(DF$Fecha, "America/Lima")`
difftime(t2, t1, units="mins") ★ NUEVO	Diferencia entre dos fechas en las unidades elegidas.	`difftime(DF$Fecha[2], DF$Fecha[1], units = "mins")`
range(DF$Fecha) ★ NUEVO	Fecha mínima y máxima (periodo de recolección).	`range(DF$Fecha)`

6. MANIPULACIÓN CON DPLYR (library(dplyr))

El operador `%>%` (pipe) significa "y luego": pasa `DF` como primer argumento de la siguiente función. `DF %>% filter(...)` es lo mismo que `filter(DF, ...)`.

Función / Comando	¿Qué hace? (uso exacto)	Ejemplo con DF
filter(DF, condición)	Filtra filas que cumplen una regla.	`DF %>% filter(Sexo == "hembra")`
arrange(DF, desc(Var))	Ordena de mayor a menor (sin desc: de menor a mayor).	`DF %>% arrange(desc(Peso))`
mutate(DF, nueva = fórmula)	Crea o sobrescribe una columna.	`DF <- DF %>% mutate(PesoKg = Peso / 1000)`
select(DF, Var1, Var2)	Se queda con las columnas indicadas (con `-Var` las quita).	`DF %>% select(Raza, Peso)` `DF %>% select(-Reporte)`
count(DF, Var) ★ NUEVO	Tabla de frecuencias de una variable (sort = TRUE ordena).	`DF %>% count(Raza, sort = TRUE)`
group_by() + summarise() ★ NUEVO	Calcula estadísticos por grupo.	`DF %>% group_by(Equipo) %>% summarise(n = n(), Prom = mean(Peso, na.rm = TRUE))`
n() ★ NUEVO	Cuenta filas dentro de summarise / mutate.	`summarise(n = n())`
distinct(DF, Var) ★ NUEVO	Filas (o valores) únicos.	`DF %>% distinct(Raza)`
case_when() ★ NUEVO	Crea categorías con varias condiciones.	`DF %>% mutate(G = case_when( Peso < 500 ~ "liviano", Peso < 1000 ~ "medio", TRUE ~ "pesado"))`

Función / Comando	¿Qué hace? (uso exacto)	Ejemplo con DF
`if_else(cond, sí, no)` ★ NUEVO	Condición simple de dos resultados.	`mutate(Adulto = if_else(Edad >= 12, "sí", "no"))`
`rename(DF, nuevo = viejo)` ★ NUEVO	Cambia el nombre de una columna.	`DF %>% rename(Kg = PesoKg)`

7. TRABAJO CON DOS BASES (DFF y DFR)

Función / Comando	¿Qué hace? (uso exacto)	Ejemplo con DF
`setdiff(names(DFF), names(DFR))` ★ NUEVO	Columnas que están en DFF y no en DFR (antes de unir, deben coincidir).	`setdiff(names(DFF), names(DFR))`
`bind_rows(DFF, DFR)` ★ NUEVO	Une las bases UNA DEBAJO DE OTRA (apila filas). `.id` crea una columna que indica el origen.	`DF <- bind_rows(DFF, DFR, .id = "Origen")`
`left_join(DFF, DFR, by = "ID")` ★ NUEVO	Une COLUMNAS por una clave: mantiene todas las filas de DFF.	`DF <- left_join(DFF, DFR, by = "ID")`
`inner_join(DFF, DFR, by = "ID")` ★ NUEVO	Solo filas cuya clave está en ambas bases.	`inner_join(DFF, DFR, by = "ID")`
`anti_join(DFF, DFR, by = "ID")` ★ NUEVO	Filas de DFF que NO tienen pareja en DFR.	`anti_join(DFF, DFR, by = "ID")`
`nrow(DFF) + nrow(DFR)` ★ NUEVO	Comprobar que bind_rows no perdió filas.	`nrow(DFF) + nrow(DFR) == nrow(DF)`

8. ESTADÍSTICA DESCRIPTIVA

Función / Comando	¿Qué hace? (uso exacto)	Ejemplo con DF
`mean(DF$Var, na.rm=TRUE)`	Promedio. Usar si CV < 30% y no hay sesgo fuerte.	`round(mean(DF$Peso, na.rm = TRUE), 2)`
`median(DF$Var, na.rm=TRUE)`	Mediana. Usar si hay sesgo, atípicos o CV > 30%.	`round(median(DF$Peso, na.rm = TRUE), 2)`
`table(DF$Var)`	Tabla de frecuencias (ignora NA por defecto).	`table(DF$Talla)`

`TP <- table(DF$Var)` ★ NUEVO	Guarda los CONTEOS por categoría (ej. corredoras por país). Luego mean/sd sobre TP dan el promedio y la sd del número por categoría.	`TP <- table(DF$País)`
		`round(mean(TP), 2)  # 2`
		`round(sd(TP), 2)  # 1.51`

`table(x, useNA = "ifany")` ★ NUEVO	Tabla de frecuencias que también cuenta los NA.	`table(DF$Talla, useNA = "ifany")`
`names(sort(table(x), decreasing=TRUE))[1]` ★ NUEVO	MODA directa (categoría más frecuente).	`names(sort(table(DF$Color), decreasing = TRUE))[1]`
`prop.table(table(x))` ★ NUEVO	Frecuencias relativas (proporciones). ×100 para %.	`round(prop.table(table(DF$Sexo))*100, 2)`
`table(x, y)` ★ NUEVO	Tabla cruzada de dos variables categóricas.	`table(DF$Raza, DF$Sexo)`
`sd(DF$Var, na.rm=TRUE)`	Desviación estándar.	`round(sd(DF$Peso, na.rm = TRUE), 2)`
`var(DF$Var, na.rm=TRUE)` ★ NUEVO	Varianza (= sd^2).	`round(var(DF$Peso, na.rm = TRUE), 2)`

Función / Comando	¿Qué hace? (uso exacto)	Ejemplo con DF
sd/mean ★ NUEVO	CV como proporción (sin ×100). Úsalo si el enunciado no pide porcentaje.	`round(sd(TP) / mean(TP), 2)  # 0.75`
sd/mean*100 ★ NUEVO	Coeficiente de variación (CV %). Decide media vs mediana.	`round(sd(DF$Peso, na.rm=TRUE) / mean(DF$Peso, na.rm=TRUE) * 100, 2)` `# 20.36`
IQR(DF$Var, na.rm=TRUE)	Rango intercuartil (Q3 – Q1). Acompaña a la mediana.	`round(IQR(DF$Peso, na.rm = TRUE), 2)`
min(), max(), range() ★ NUEVO	Mínimo, máximo y ambos a la vez.	`range(DF$Edad, na.rm = TRUE)`
rowMeans(DF[, c("V1","V2")])	Promedio fila por fila de varias columnas.	`rowMeans(DF[, c("Largo","Cintura")], na.rm = TRUE)`
cor(x, y, use=...) ★ NUEVO	Correlación entre dos numéricas.	`round(cor(DF$Peso, DF$Largo, use = "complete.obs"), 2)`
round(cálculo, 2)	Redondea a 2 decimales. ¡ÚSALO SIEMPRE en toda respuesta numérica!	`round(mean(DF$Peso, na.rm=TRUE), 2)`

9. VALORES ATÍPICOS (OUTLIERS)

Función / Comando	¿Qué hace? (uso exacto)	Ejemplo con DF
quantile(x, 0.25, na.rm=TRUE)	Cuartil 1 (Q1): deja el 25% de datos por debajo.	`Q1 <- quantile(DF$Peso, 0.25, na.rm = TRUE)`
quantile(x, 0.75, na.rm=TRUE)	Cuartil 3 (Q3): deja el 75% de datos por debajo.	`Q3 <- quantile(DF$Peso, 0.75, na.rm = TRUE)`
Límites de Tukey ★ NUEVO	Atípico si x < Q1 − 1.5·RIC o x > Q3 + 1.5·RIC.	`RIC <- IQR(DF$Peso, na.rm = TRUE)` `LI <- Q1 - 1.5*RIC` `LS <- Q3 + 1.5*RIC`
sum(x < LI | x > LS, na.rm=TRUE) ★ NUEVO	Cuenta cuántos atípicos hay.	`sum(DF$Peso < LI \| DF$Peso > LS, na.rm = TRUE)  # 183`
boxplot.stats(x)$out ★ NUEVO	Atajo: devuelve directamente los valores atípicos.	`length(boxplot.stats(DF$Peso)$out)`
boxplot(DF$Var) ★ NUEVO	Diagrama de caja para ver atípicos.	`boxplot(DF$Peso ~ DF$Raza)`
hist(DF$Var) ★ NUEVO	Histograma para ver forma y sesgo.	`hist(DF$Tiempo, breaks = 50)`

10. PROBABILIDADES EMPÍRICAS Y OPERADORES LÓGICOS

Probabilidad empírica de un evento = proporción de filas que lo cumplen. En R: `round(mean(condición, na.rm = TRUE), 2)`, porque la media de TRUE/FALSE es la proporción de TRUE.

Notación	Significado	En R
P(A)	Ocurre A	`round(mean(A, na.rm = TRUE), 2)`
A ∩ B	A **y** B	`A & B`
A ∪ B	A **o** B (o ambos)	`A | B`
Ā (A con barra)	**No** A (complemento)	`!A`
Barra sobre (A ∩ B)	No ocurren A y B a la vez	`!(A & B)`
P(A | B)	A **dado** B (condicional)	`mean(A[B], na.rm = TRUE)`
Ā ∩ B̄	Ni A ni B (= barra sobre A ∪ B)	`!A & !B` ≡ `!(A | B)`

Función / Comando	¿Qué hace? (uso exacto)	Ejemplo con DF
mean(condición, na.rm=TRUE) ★ NUEVO	Probabilidad empírica: proporción de TRUE ignorando las filas con NA.	`round(mean(DF$Streamers > 10000, na.rm = TRUE), 2)  # 0.51`
sum(condición, na.rm=TRUE) ★ NUEVO	Cuántas filas cumplen la condición (casos favorables).	`sum(DF$Mes == "Mayo")`
A <- condición ★ NUEVO	Guarda un evento como vector lógico para combinarlo después.	`X <- DF$Streamers > 10000` `Y <- DF$PromCan < 200`
& , | , ! ★ NUEVO	Y, O, NO. Los paréntesis definen qué se niega.	`round(mean(X & Y, na.rm = TRUE), 2)` `round(mean(!(X & Y) & !(U | O), na.rm = TRUE), 2)`
== , != , > , < , >= , <= ★ NUEVO	Igual, distinto, mayor, menor... (para textos usa comillas).	`U <- DF$Mes == "Mayo"` `O <- DF$` `` `Año` `` ` != 2020`
DF$`Var con tilde` ★ NUEVO	Las comillas invertidas permiten usar nombres con ñ, tildes o espacios.	`table(DF$` `` `Año` `` `)`
xor(A, B) ★ NUEVO	EXACTAMENTE uno de los dos (A o B pero no ambos).	`round(mean(xor(X, Y), na.rm = TRUE), 2)`
Trampa del | ★ NUEVO	En el ENUNCIADO P(A | B) = A DADO B (condicional → corchetes). En R, A | B = A O B (unión). Palabras: y/pero/ambos → &; o/al menos uno → |; no/ni/excepto → !; dado que/entre los que/sabiendo → A[B].	`mean(X[Y], na.rm=TRUE)  # P(X | Y)` `mean(X | Y, na.rm=TRUE)  # P(X ∪ Y)`
mean(A[B], na.rm=TRUE) ★ NUEVO	Probabilidad CONDICIONAL P(A | B): proporción de A solo entre las filas donde ocurre B.	`round(mean(X[Y], na.rm = TRUE), 2)` `# P(X | Y) = 0.24`
sum(A & B)/sum(B) ★ NUEVO	Misma P(A | B) con la fórmula P(A ∩ B) / P(B) en conteos.	`round(sum(X & Y, na.rm = TRUE) / sum(Y, na.rm = TRUE), 2)`
Independencia ★ NUEVO	A y B son independientes si P(A | B) ≈ P(A) (iguales al redondear a 2 decimales). Si difieren: dependientes. Disjuntos = no pueden ocurrir juntos: P(A ∩ B) = 0.	`round(mean(X[Y], na.rm=TRUE), 2) == round(mean(X, na.rm=TRUE), 2)`
Probabilidad total ★ NUEVO	Si A, B, C reparten todos los casos: P(V) = P(V|A)·P(A) + P(V|B)·P(B) + P(V|C)·P(C). Se usa con datos TEÓRICOS (de un experto/modelo).	`PV <- 0.90*0.025 + 0.30*0.475 + 0.05*0.500` `round(PV, 2)  # 0.19`
Teorema de Bayes ★ NUEVO	Invierte la condicional: P(A|V) = P(V|A)·P(A) / P(V).	`round(0.90*0.025 / PV, 2)  # 0.12`

Rangos con & ★ NUEVO	Eventos por tramos de una variable (ej. posición entre 6 y 100).	`A <- DF$Pos <= 5`
		`B <- DF$Pos > 5 & DF$Pos <= 100`
		`C <- DF$Pos > 100`

filter(!is.na(Var)) ★ NUEVO	Quita los NA de una variable antes de calcular (otra forma de tratar los NA).	`DF %>% filter(!is.na(PromCan))`

11. VARIABLES ALEATORIAS CONTINUAS (FUNCIÓN DE DENSIDAD)

Receta: 1) define la densidad por partes con `ifelse`; 2) halla k para que el área total sea 1; 3) integra para probabilidades, valor esperado y desviación estándar.

Función / Comando	¿Qué hace? (uso exacto)	Ejemplo con DF
`function(x) ...` ★ NUEVO	Crea tu propia función en R (aquí, la densidad).	`g <- function(x) x^2`

`ifelse(cond, sí, no)` ★ NUEVO	Define una función POR PARTES (se pueden anidar varios ifelse). Funciona con vectores.	`g <- function(x) ifelse(x > 0 & x <= 1,`
		`  4 - 3*(1 - x)^2,`
		`  ifelse(x > 1 & x <= 3, (x - 3)^2, 0))`

`integrate(f, a, b)$value` ★ NUEVO	Integral de f entre a y b (usa -Inf / Inf si hace falta). `$value` extrae el número.	`integrate(g, 0, 3)$value  # 5.67`

`k = 1 / ∫ g` ★ NUEVO	Constante de normalización: el área bajo la densidad debe ser 1.	`k <- 1 / integrate(g, 0, 3)$value`
		`f <- function(x) k * g(x)`
		`round(k, 2)  # 0.18`

`P(a < X ≤ b) = ∫ f` ★ NUEVO	Probabilidad teórica = área bajo f entre a y b.	`round(integrate(f, 1, 3)$value, 2)` `# P(X > 1) = 0.47`
`E(X) = ∫ x·f(x)` ★ NUEVO	Valor esperado (media teórica).	`E <- integrate(function(x) x*f(x),` `  0, 3)$value`

`sd = √(E(X²) − E(X)²)` ★ NUEVO	Desviación estándar teórica.	`E2 <- integrate(function(x) x^2*f(x),`
		`  0, 3)$value`
		`S <- sqrt(E2 - E^2)`

`sqrt(x)` ★ NUEVO	Raíz cuadrada.	`sqrt(25)  # 5`
`uniroot(f, c(a, b))$root` ★ NUEVO	Halla x tal que f(x) = 0. Sirve para la MEDIANA teórica: F(m) = 0.5.	`uniroot(function(m) integrate(f, 0, m)$value` `  - 0.5, c(0, 3))$root  # 0.96`
`CV = S / E * 100` ★ NUEVO	Si CV > 30% → usar la MEDIANA como medida de centralidad; si CV < 30% → el PROMEDIO.	`round(S / E * 100, 2)  # 55.52 -> mediana`

12. MODELOS DE PROBABILIDAD (DISTRIBUCIONES)

En R cada modelo tiene 4 funciones con el mismo prefijo: `d` = probabilidad puntual/densidad, `p` = acumulada P(X ≤ x), `q` = cuantil, `r` = simular. Para P(X > x) usa `lower.tail = FALSE`.

Cómo validar un modelo: estima el parámetro con la media de los datos, compara la sd y una probabilidad TEÓRICAS con las EMPÍRICAS. Si difieren mucho → el modelo NO es bueno.

Modelo	Parámetro desde la media m	Media / sd teóricas
Geométrico (R cuenta fracasos antes del 1er éxito: 0,1,2...)	`p <- 1 / (m + 1)`	media = (1−p)/p ; sd = `sqrt((1 - p) / p^2)`
Poisson	`lambda <- m`	media = λ ; sd = `sqrt(lambda)`
Binomial (n intentos)	`p <- m / n`	media = np ; sd = `sqrt(n*p*(1 - p))`
Exponencial	`rate <- 1 / m`	media = 1/rate ; sd = 1/rate (sd = media)
Normal	`mu <- m ; s <- sd(x)`	media = μ ; sd = σ
Gamma (shape, rate)	dados en el enunciado (sha, rat)	media = shape/rate ; sd = `sqrt(shape)/rate`

Función / Comando	¿Qué hace? (uso exacto)	Ejemplo con DF
`dgeom(x, p)` ★ NUEVO	P(X = x) en el modelo geométrico.	`dgeom(0, p)`
`pgeom(x, p)` ★ NUEVO	P(X ≤ x) geométrico (acumulada).	`pgeom(m, p)`
`pgeom(x, p, lower.tail=FALSE)` ★ NUEVO	P(X > x) geométrico (cola superior).	`round(pgeom(m, p,` `  lower.tail = FALSE), 2)  # 0.37`

Función / Comando	¿Qué hace? (uso exacto)	Ejemplo con DF
dpois / ppois(x, lambda) ★ NUEVO	Puntual / acumulada Poisson.	`ppois(3, lambda = 2)`
dbinom / pbinom(x, n, p) ★ NUEVO	Puntual / acumulada binomial.	`dbinom(2, size = 10, prob = 0.3)`
pexp(x, rate) ★ NUEVO	Acumulada exponencial.	`pexp(1, rate = 1/m)`
pnorm(x, mu, s) / qnorm(prob, mu, s) ★ NUEVO	Acumulada normal / cuantil normal.	`pnorm(1.96)  # 0.975` `qnorm(0.975) # 1.96`
dgamma / pgamma / qgamma ★ NUEVO	Densidad, acumulada y cuantiles del modelo GAMMA (shape, rate).	`pgamma(50, 1.92, 0.05, lower.tail = FALSE)  # P(Z ≥ 50)` `qgamma(c(.25,.5,.75), 1.92, 0.05)`
P(a < X < b) continua ★ NUEVO	Diferencia de acumuladas: F(b) − F(a).	`pgamma(50, sha, rat) - pgamma(12.5, sha, rat)  # 0.59`
IQR teórico ★ NUEVO	Q3 − Q1 del modelo con la función q.	`diff(qgamma(c(.25, .75), sha, rat))`
rgeom / rpois / rnorm(n, ...) ★ NUEVO	Simula n valores del modelo (con set.seed para repetir).	`set.seed(1); rgeom(5, p)`

13. GRÁFICOS (MUESTRA vs MODELO TEÓRICO)

Función / Comando	¿Qué hace? (uso exacto)	Ejemplo con DF
hist(x, prob = TRUE) ★ NUEVO	Histograma en escala de DENSIDAD (área = 1) para poder superponer la curva teórica.	`hist(Z, prob = TRUE, breaks = seq(0, 300, 12.5))`
seq(desde, hasta, paso) ★ NUEVO	Secuencia de números (cortes del histograma, ejes).	`seq(0, 300, 12.5)`
xlim, ylim, xlab, ylab, main ★ NUEVO	Límites de ejes, títulos de ejes y título del gráfico.	`hist(Z, xlim = c(0, 300), main = "Gráfica")`
boxplot(x, horizontal=TRUE, add=TRUE) ★ NUEVO	Diagrama de caja horizontal añadido al gráfico actual (at = altura).	`boxplot(Z, horizontal = TRUE, add = TRUE, at = -0.0015)`
lines(x, y, col, lwd) ★ NUEVO	Dibuja una curva (ej. la densidad teórica) sobre el gráfico.	`lines(0:300, dgamma(0:300, sha, rat), col = "red", lwd = 2)`
abline(v = valores) ★ NUEVO	Líneas verticales (v) u horizontales (h): medias, cuartiles...	`abline(v = mean(Z, na.rm = TRUE))`
legend("topright", legend = ...) ★ NUEVO	Agrega la leyenda.	`legend("topright", legend = c("teórica"), col = "red")`
paste(..., sep = "") ★ NUEVO	Une textos y números.	`paste("gamma(", sha, ")", sep = "")`
Leer el gráfico ★ NUEVO	Cola larga a la derecha = sesgo a la derecha (media > mediana). En el boxplot: bordes de la caja = Q1 y Q3 empíricos. Si Q3 > c, entonces P(X > c) ≥ 0.25.	—

14. CONCEPTOS CLAVE: LIMPIA vs PROLIJA

Concepto	Significado	Cómo se verifica en R
Base LIMPIA	Los valores respetan la libreta de códigos (categorías válidas, rangos correctos, tipos correctos).	`unique()`, `range()`, `%in%`, `summary()`
Base PROLIJA (tidy)	1 variable por columna, 1 observación por fila, 1 valor por celda.	Buscar celdas con varios datos juntos y separarlas con `separate()`
Observación INCOMPLETA	Fila con al menos un NA.	`sum(!complete.cases(DF))`

15. EJERCICIOS RESUELTOS

T2 · Pregunta 1 — Exploración de la base Cuy (Cuytopia)

```
library(readr); library(dplyr)
DF <- read_csv("Cuy_InicialYPiloto.csv")

dim(DF)                      # 8755 observaciones de 13 variables
sapply(DF, class)            # tipo de cada variable
sapply(DF, n_distinct)       # valores distintos por variable

n_distinct(DF$Tiempo)        # 8574 -> casi un valor distinto por fila

head(DF$Reporte)             # 3 datos en 1 celda (Ser. No., Date, Status)
head(DF$Fecha)               # fecha y hora juntas

sum(!complete.cases(DF))     # 4748 observaciones incompletas
```

Espacio en blanco	Respuesta	Justificación
Variables categóricas nominales	**6**	Raza, Sexo, Color, Reporte, Equipo, Error
Variables numéricas	**5**	Peso, Largo, Cintura, Edad, Tiempo (6+1+1+5 = 13)
Numéricas discretas	**4**	Peso (2 decimales), Largo, Cintura y Edad (enteros): continuas pero discretizadas al registrarse
Describirla con…	**una tabla de frecuencias**	`n_distinct(DF$Tiempo)` = 8574
…esperamos ver una tabla	**muy grande con pocos valores repetidos**	8574 valores distintos en 8755 filas: casi cada valor aparece una sola vez
La base de datos está	**limpia**	El enunciado dice que fue procesada y limpiada según la libreta de códigos (pero no es prolija)
Variable que podría no estar prolija	**Fecha**	Guarda fecha y hora en una misma celda; según el análisis habría que separarlas
Observaciones incompletas	**4748**	`sum(!complete.cases(DF))`

EF · Pregunta 1 — Probabilidades empíricas (Twitch)

```
library(readr)
DF <- read_csv("TwitchAF.csv")

X <- DF$Streamers > 10000   # más de 10000 streamers
Y <- DF$PromCan < 200       # menos de 200 canales en promedio
U <- DF$Mes == "Mayo"       # mes de mayo
O <- DF$`Año` != 2020       # no es del año 2020

round(mean(X, na.rm = TRUE), 2)                    # P(X)       0.51
round(mean(Y, na.rm = TRUE), 2)                    # P(Y)       0.64
round(mean(U, na.rm = TRUE), 2)                    # P(U)       0.09
round(mean(O, na.rm = TRUE), 2)                    # P(O)       0.89
round(mean(X & Y, na.rm = TRUE), 2)                # P(X ∩ Y)   0.13
round(mean(U | O, na.rm = TRUE), 2)                # P(U ∪ O)   0.89
round(mean(!U & !Y, na.rm = TRUE), 2)              # P(Ū ∩ Ȳ)   0.32
round(mean(!(X & Y) & !(U | O), na.rm = TRUE), 2)  # último     0.08
```

EF · Pregunta 2 — Probabilidad condicional e independencia (Twitch)

```
X <- DF$Streamers > 10000   # más de 10000 streamers
Y <- DF$PromCan < 200       # menos de 200 canales en promedio
E <- DF$Mes == "Enero"      # mes de enero
M <- DF$`Año` == 2024       # año 2024

round(mean(X[Y], na.rm = TRUE), 2)    # P(X | Y)   0.24
round(mean(X, na.rm = TRUE), 2)       # P(X)       0.51  -> distintas: DEPENDIENTES
```

```
round(mean(E[!M], na.rm = TRUE), 2)   # P(E | no M) 0.08
round(mean(E, na.rm = TRUE), 2)       # P(E)        0.08  -> iguales: INDEPENDIENTES
```

EF · Pregunta 3 — Probabilidad total y Bayes: teórico vs empírico (Twitch)

```
# Modelo TEÓRICO del experto (probabilidad total y Bayes)
PV <- 0.90*0.025 + 0.30*0.475 + 0.05*0.500
round(PV, 2)                    # P(V)      0.19
round(0.90*0.025 / PV, 2)       # P(A | V)  0.12

# Probabilidades EMPÍRICAS con DF
A <- DF$Pos <= 5                # entre las cinco mejores
V <- DF$Streamers >= 20000      # al menos 20000 streamers (>=)
round(mean(V, na.rm = TRUE), 2)      # P(V)      0.31
round(mean(A[V], na.rm = TRUE), 2)   # P(A | V)  0.21
# Experto 0.19 < observado 0.31 -> escenario MÁS PESIMISTA que el observado
```

EF · Pregunta 4 — Función de densidad por partes (Twitch)

```
g <- function(x) ifelse(x > 0 & x <= 1, 4 - 3*(1 - x)^2,
                 ifelse(x > 1 & x <= 3, (x - 3)^2, 0))
k <- 1 / integrate(g, 0, 3)$value
f <- function(x) k * g(x)

round(k, 2)                                          # k          0.18
round(integrate(f, 1, 3)$value, 2)                   # P(X > 1)   0.47
E  <- integrate(function(x) x * f(x), 0, 3)$value
E2 <- integrate(function(x) x^2 * f(x), 0, 3)$value
S  <- sqrt(E2 - E^2)
round(E, 2)                                          # E(X)       1.01 horas
round(S, 2)                                          # sd         0.56 horas
round(S / E * 100, 2)                                # CV 55.52% > 30% -> MEDIANA
```

EF · Pregunta 5 — Validar un modelo geométrico (Twitch)

```
m <- mean(DF$Streamers, na.rm = TRUE)
p <- 1 / (m + 1)                    # parámetro geométrico

round(m, 2)                                           # media        34054.58
round(sqrt((1 - p) / p^2), 2)                         # sd teórica   34055.08
round(pgeom(m, p, lower.tail = FALSE), 2)             # P(X > media) 0.37

round(sd(DF$Streamers, na.rm = TRUE), 2)              # sd empírica  61174.47
round(mean(DF$Streamers > m, na.rm = TRUE), 2)        # P empírica   0.23
# sd teórica << sd empírica -> NO ES BUENO: variabilidad mucho menor
```

EF · Pregunta 6 — Gráfica comparativa: muestra vs modelo gamma (Twitch)

```
sha <- 1.92; rat <- 0.05
Z <- DF$PromEsp / DF$PromCan

round(mean(Z, na.rm = TRUE), 2); round(median(Z, na.rm = TRUE), 2)  # 39.56 > 28.68 ->
sesgo derecha
round(IQR(Z, na.rm = TRUE), 2)                          # IQR empírico 35.74
round(diff(qgamma(c(.25, .75), sha, rat)), 2)          # IQR teórico  33.78 (menor)
sum(!is.na(Z)); round(max(Z, na.rm = TRUE), 2)          # 1965 y 227.79 (no se ve en la
gráfica)
round(mean(Z > 50, na.rm = TRUE), 2)                    # 0.28 >= 0.25 (Q3 emp. 53.56 >
50)
round(pgamma(50, sha, rat) - pgamma(12.5, sha, rat), 2) # 0.59 (requiere cálculo)
round(pgamma(50, sha, rat, lower.tail = FALSE), 2)      # 0.27 >= 0.25 (Q3 teo. 51.84 >
50)
# Marcar (solo con la gráfica): sesgada a la derecha; P emp(Z>50) >= 0.25; P teo(Z>=50)
>= 0.25
```

Lab 2 · Pregunta 1 — Exploración de la etapa eliminatoria (Atletismo)

```
library(readr)
DF <- read_csv("Eliminatoria-3.csv")

str(DF)                        # 32 obs. de 8 variables
nrow(DF)                       # 32 observaciones
ncol(DF)                       # 8 variables
sum(complete.cases(DF))        # 32 -> no hay NA
length(unique(DF$País))        # 16 países
sum(DF$Primeros)               # 27 primeros lugares
sum(DF$Segundos)               # 24 segundos lugares
sum(DF$Terceros)               # 25 terceros lugares
```

Lab 2 · Pregunta 2 — Corredoras por país: promedio, sd y CV

```
TP <- table(DF$País)             # número de corredoras por país
round(mean(TP), 2)               # promedio  2
round(sd(TP), 2)                 # sd        1.51
round(sd(TP) / mean(TP), 2)      # CV        0.75  (en % sería 75.28)
```